```
void f(int i,std::string const& s);

void not_oops(int some_param)
{
    char buffer[1024];
    sprintf(buffer,"%i",some_param);
    std::thread t(f,3,std::string(buffer));
    t.detach();
}
```

Using std::string avoids dangling pointer

In this case, the problem is that you were relying on the implicit conversion of the pointer to the buffer into the `std::string` object expected as a function parameter, because the `std::thread` constructor copies the supplied values as is, without converting to the expected argument type.

It's also possible to get the reverse scenario: the object is copied, and what you wanted was a reference. This might happen if the thread is updating a data structure that's passed in by reference, for example:

```
void update_data_for_widget(widget_id w,widget_data& data);    ❶

void oops_again(widget_id w)
{
    widget_data data;
    std::thread t(update_data_for_widget,w,data);    ❷
    display_status();
    t.join();
    process_widget_data(data);    ❸
}
```

Although `update_data_for_widget` ❶ expects the second parameter to be passed by reference, the `std::thread` constructor ❷ doesn't know that; it's oblivious to the types of the arguments expected by the function and blindly copies the supplied values. When it calls `update_data_for_widget`, it will end up passing a reference to the internal copy of `data` and not a reference to `data` itself. Consequently, when the thread finishes, these updates will be discarded as the internal copies of the supplied arguments are destroyed, and `process_widget_data` will be passed an unchanged `data` ❸ rather than a correctly updated version. For those of you familiar with `std::bind`, the solution will be readily apparent: you need to wrap the